

Soil Moisture Forecasting

Semester: HS26

Project Title	Soil Moisture Forecasting
Course	MLOPS H26
Author	Natalie Sumbo Filipe
Repository	github.com/sky150/soil-moisture-forecasting-uk

Short project description: This live ML system forecasts daily field-scale soil moisture at one agricultural station in the UK's COSMOS-UK network. The system automatically ingests soil and weather observations, retrain a regression model and displays forecasts alongside subsequent measurements in a cloud dashboard.

1 Problem statement

Problem and intended users: Soil moisture influences plant growth, water availability and how soils respond to rainfall. Monitoring only describes existing conditions. Short-term forecasts help agricultural researchers and farm managers to anticipate drying and wetting at a monitored site.

Prediction: Predict the average soil moisture for a future calendar day, expressed as volumetric water content in percent (`cosmos_vwc`).

Horizon: Tomorrow's daily average, using measurements through yesterday. The API supplies completed daily observations once per day. This gives a two-calendar-day gap between the latest measurements and the target date.

Scope: One agricultural COSMOS-UK station in the UK and one regression target. Initially use 2023 onward plus new daily observations. Earlier history can be added if useful. BALRD is a tested candidate with records starting in May 2014, so 2023 is a chosen starting window, not the limit of available history. The final station will be selected before training based on coverage and quality of the data.

Evaluation: MAE over daily test forecasts, computed as $MAE = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i|$, where N is the number of evaluated days, $\hat{y}_i$ the forecast and y_i the observed daily moisture.

Baseline: Persistence repeats the latest available soil moisture measurement for the target day. For baseline evaluation, compute its MAE directly from historical observations before model training, using the same forecast horizon and chronological test dates as the ML model.

Success criterion: Aim for a test MAE at least 10% lower than the persistence baseline.

2 Originality & motivation

I am interested in agriculture and in how machine learning can support practical environmental decisions. In my free time, I have built monitoring projects for plants at home. This project extends that interest from observing current plant conditions to forecasting soil moisture using an established agricultural monitoring site. I checked the HSLU FS26 Projects and the KTH ID2223 2026. Those lists include projects focused on forecasting air quality, solar output, lake water levels and water temperature. This project differs by forecasting sensor-derived agricultural soil moisture. COSMOS-UK is operated by the UK Centre for Ecology & Hydrology, an independent research institute. Its open data support farming, drought and flood research, and water-resource management.

3 Data source & features

Live source and growth: UKCEH's COSMOS-UK REST API supplies soil and weather observations without an API key under the Open Government Licence. The daily collection updates once per day through yesterday. The dataset grows by approximately one record per day per station. One tested station, Balruddery (BALRD), has records starting in May 2014 according to its metadata. Initially backfill 2023 onward in yearly requests: the tested BALRD endpoint returned 1,370 daily records through 1 October 2026, with no missing `cosmos_vwc` values.

Label: Daily field-scale soil moisture (`cosmos_vwc`), in percent, on the forecast target date. The label comes from subsequent COSMOS-UK observations derived from cosmic-ray measurements, rather than a rule applied to the input features.

Features: The model uses recent soil moisture, including the latest value 1-, 3- and 7-day lags, and 3- and 7-day averages. It also uses 3- and 7-day rainfall totals (`precip`), air temperature (`ta`), relative humidity (`rh`), soil temperature at 20 cm (`stp_tsoil20`) and day of year to capture seasonality. These features come from the same API and are available when the forecast is issued.

Data quality: Remove duplicate records and columns with no observations, validate measurement ranges, and exclude examples with missing targets. Preserve calendar dates for correct lag features and refresh recent data to capture provider corrections.

Leakage: Features use only measurements available when the forecast is issued. Historical data may include later corrections, so live evaluation retains the original inputs and forecasts.

Split: Train on 2023–2024 data, tune the model on 2025 and evaluate it on 2026 data through 1 October. Assign examples by their target date to keep future labels out of the training period. The test period is reserved for final evaluation and is not used for model selection. Evaluate new live forecasts separately as their observed values become available.

4 System design

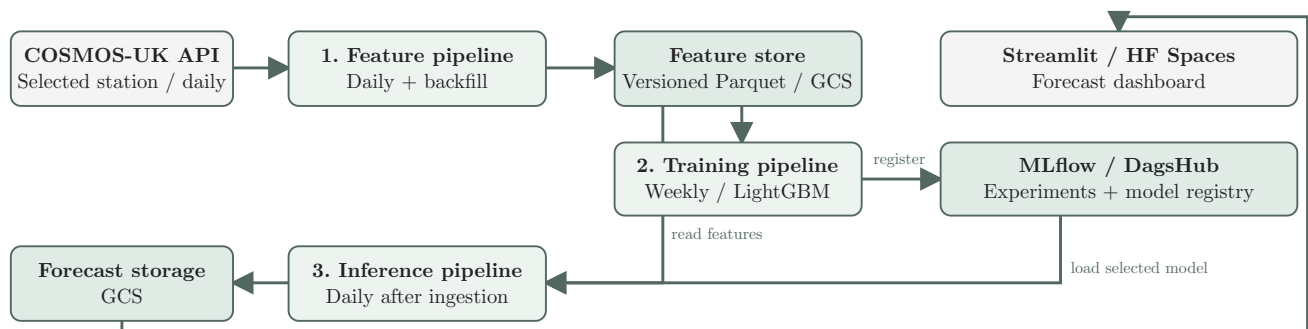


Figure 1: FTI architecture

Feature store: Versioned Parquet files on Google Cloud Storage (GCS) hold cleaned observations and engineered features for training and inference, providing a lightweight feature store suited to a small daily dataset.

Training and experiment tracking: LightGBM handles tabular regression. Hosted MLflow on DagsHub tracks MAE against persistence and registers model versions. The model with the lowest MAE on the shared validation set is selected for inference. Validation uses 2025 initially, then the latest 60 labelled days after the test period.

Orchestration: GitHub Actions runs Dockerised Python pipelines for daily ingestion and backfill, weekly retraining, and inference after successful ingestion. This provides simple scheduled batch processing integrated with CI.

Serving: The inference pipeline loads the selected model from the registry and stores daily forecasts in GCS. A Dockerised Streamlit dashboard on Hugging Face Spaces displays forecasts, observations and model version, providing a straightforward public-facing cloud UI.

Monitoring: Track data freshness, missing values, feature drift and forecast MAE once observations arrive. Docker, pinned dependencies and CI tests support reproducibility.

Stretch (optional): Additional agricultural stations

Repository: The GitHub repository is public. The proposal will be committed as `docs/proposal.pdf`.